

COIT20258 Software Engineering

Week 5

Design & Implementation

Topics

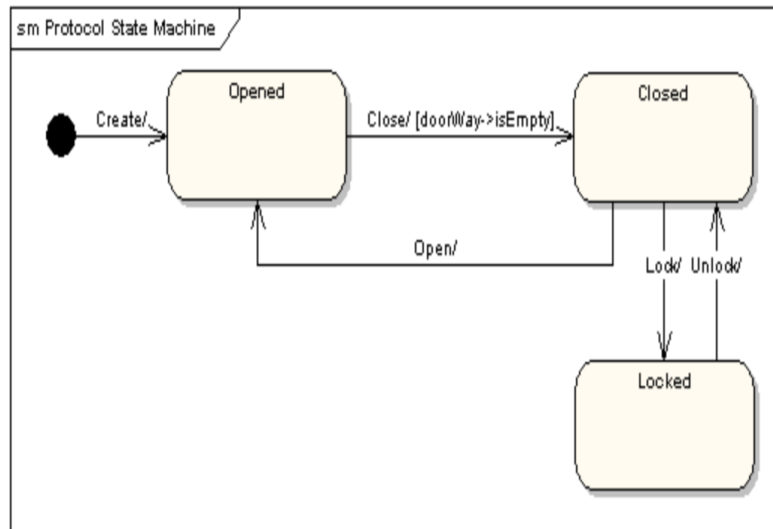
- ✧ Object-oriented (O-O) design using UML
- ✧ Design patterns
- ✧ Implementation issues

State Diagrams

- ✧ State diagrams are used to show how objects *respond to different service requests* & the *state transitions* triggered by these requests
- ✧ State diagrams are useful high-level models of a system or an object's *run-time behavior*
- ✧ You don't usually need a state diagram for all of the objects in the system. Many of the objects in a system are relatively simple & a state model adds unnecessary detail to the design.

Example: State Diagrams

As an example, the following state machine diagram shows the states that a door goes through during its lifetime.

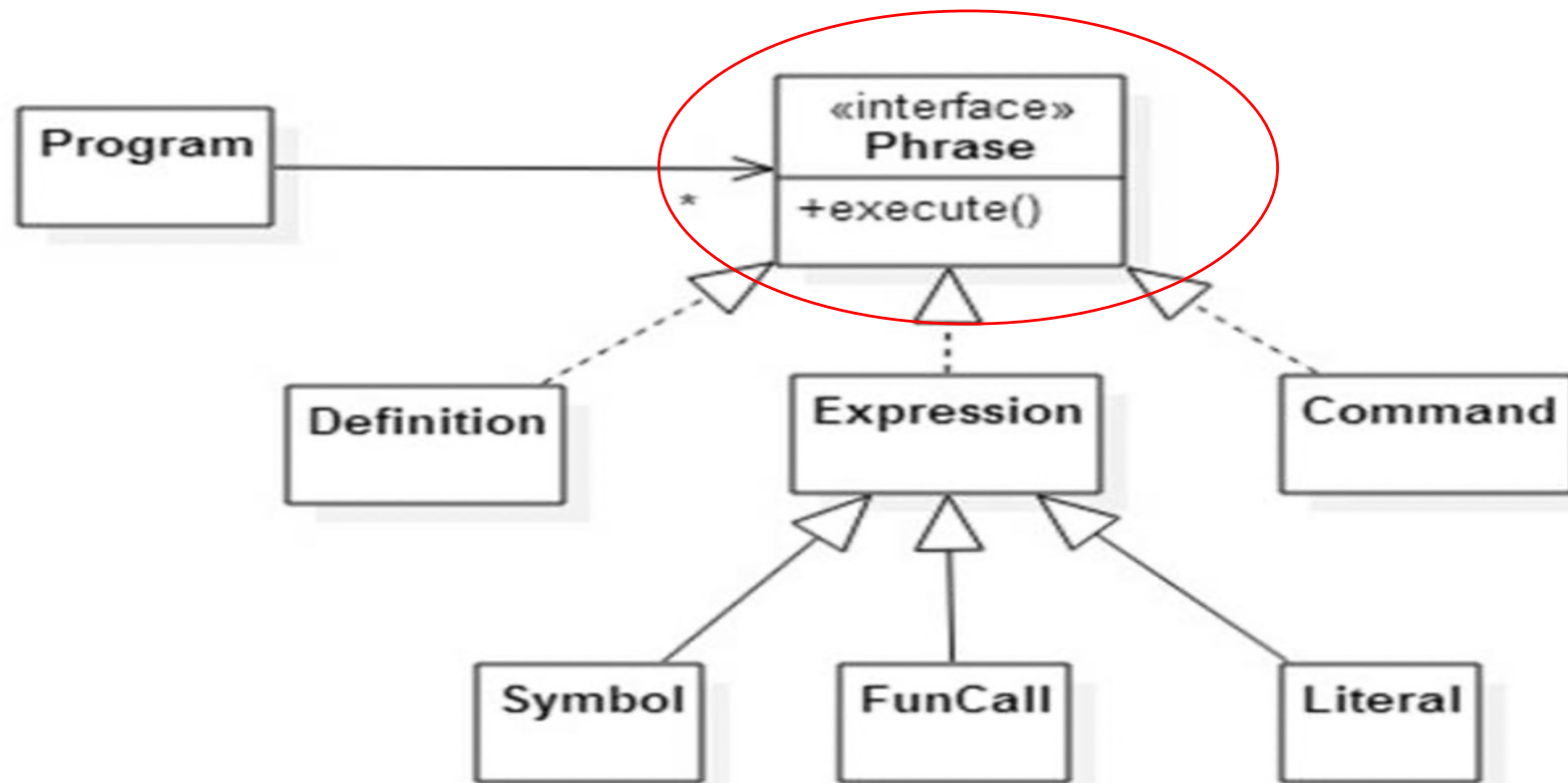


The door can be in one of three states: "Opened", "Closed" or "Locked". It can respond to the events Open, Close, Lock and Unlock. Notice that not all events are valid in all states; for example, if a door is opened, you cannot lock it until you close it. Also notice that a state transition can have a guard condition attached: if the door is Opened, it can only respond to the Close event if the condition `doorWay->isEmpty` is fulfilled. The syntax and conventions used in state machine diagrams will be discussed in full in the following sections.

Interface Specification

- ✧ Object interfaces should be specified so that the objects & other components can be *designed in parallel*
- ✧ Objects may have several interfaces which are viewpoints on the methods provided
- ✧ Designers should avoid designing data representation in an interface design, as attributes are not defined in an interface specification
- ✧ UML uses *class diagrams* for interface specification (in this case, no attribute section)

Example: Interface Specification



Design Patterns

- ✧ A design pattern is a way of *reusing abstract knowledge* about a problem & its solution
- ✧ A pattern is a description of the *problem* & the essence of its *solution*
- ✧ It should be sufficiently abstract to be *reused* in different settings
- ✧ Pattern descriptions usually make use of O-O characteristics such as *inheritance* & *polymorphism* (i.e., allows a specific routine to use variables of different types at different times)

Patterns

✧ *Patterns & Pattern Languages are ways to describe best practices, good designs, & capture experience in a way that it is possible for others to **reuse** this experience*

The Observer Pattern

✧ Name

- Observer

✧ Description

- Separates the display of object state from the object itself

✧ Problem description

- Used when *multiple displays of state* are needed

✧ Solution description

- See slide with UML description

✧ Consequences

- Optimizations to enhance display performance are impractical

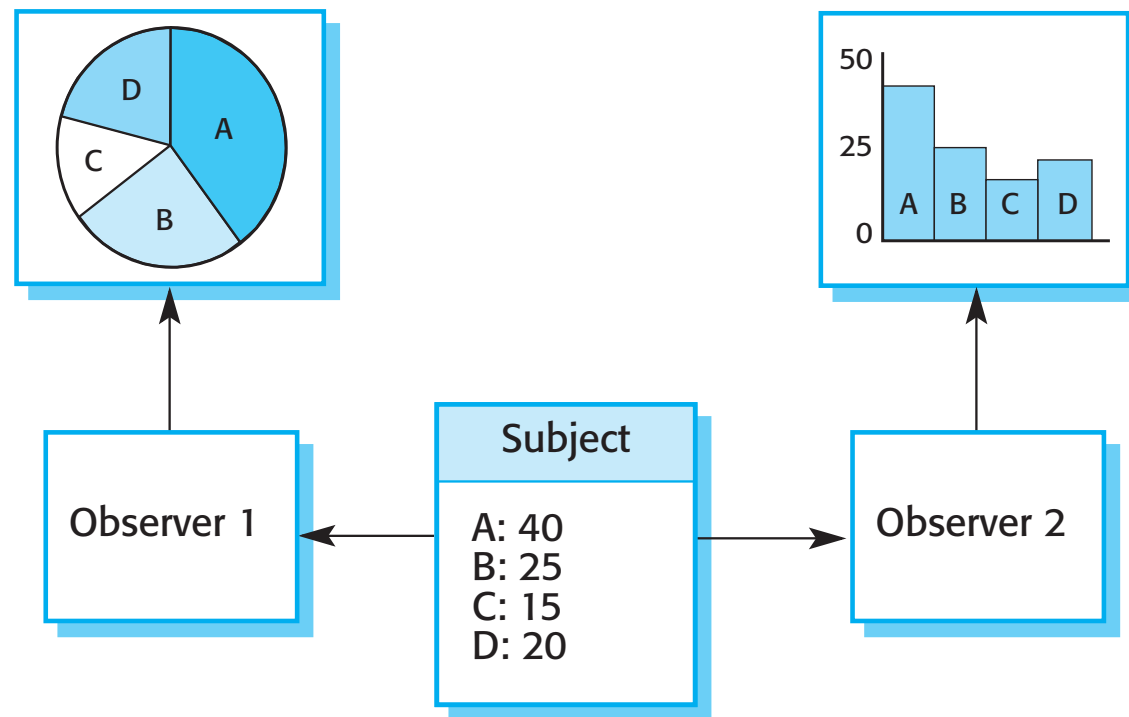
... The Observer Pattern

Pattern name	Observer
Description	Separates the display of the state of an object from the object itself & allows <i>alternative displays</i> to be provided. When the object state changes, all displays are automatically notified & updated to reflect the change.
Problem description	In many situations, you need to provide multiple displays of state information, such as a graphical display & a tabular display. Not all of these may be known when the information is specified. All alternative presentations should support interaction &, when the state is changed, all displays must be updated. This pattern may be used in all situations where more than one display format for state information is required & where it is not necessary for the object that maintains the state information to know about the specific display formats used.

... The Observer Pattern

Pattern name	Observer
Solution description	This involves two abstract objects, <i>Subject</i> & <i>Observer</i>, & two concrete objects, <i>ConcreteSubject</i> & <i>ConcreteObserver</i>, which inherit the attributes of the related abstract objects. The abstract objects include general operations that are applicable in all situations. The state to be displayed is maintained in <i>ConcreteSubject</i>, which inherits operations from <i>Subject</i> allowing it to add & remove <i>Observers</i> (each observer corresponds to a display) & to issue a notification when the state has changed. The <i>ConcreteObserver</i> maintains a copy of the state of <i>ConcreteSubject</i> & implements the <i>Update()</i> interface of <i>Observer</i> that allows these copies to be kept in step. The <i>ConcreteObserver</i> automatically displays the state & reflects changes whenever the state is updated.
Consequences	The subject only knows the abstract <i>Observer</i> & does not know details of the concrete class. Therefore, there is minimal coupling between these objects. Because of this lack of knowledge, optimizations that enhance display performance are impractical. Changes to the subject may cause a set of linked updates to observers to be generated, some of which may not be necessary.

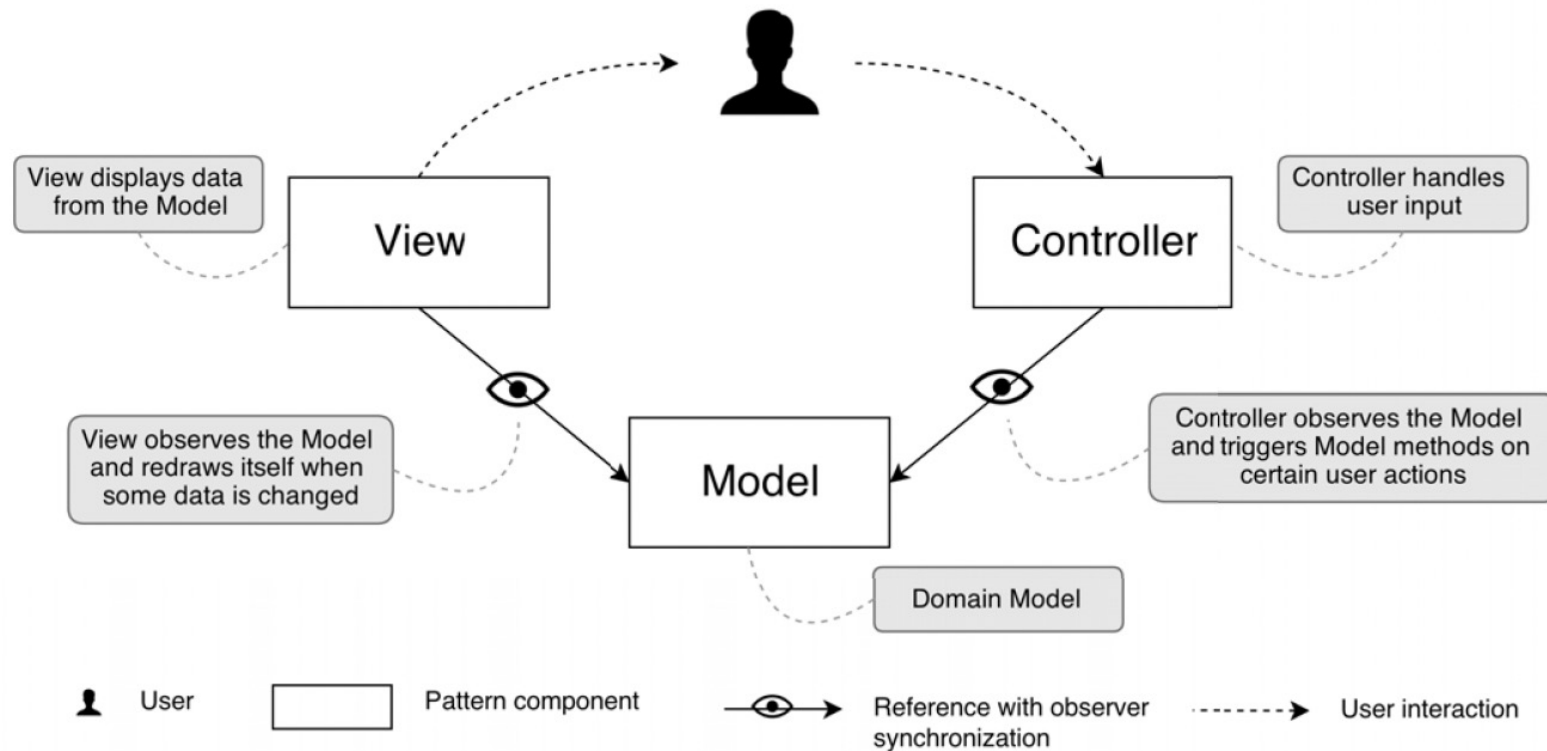
Multiple Displays Using the Observer Pattern



MVC Design Pattern

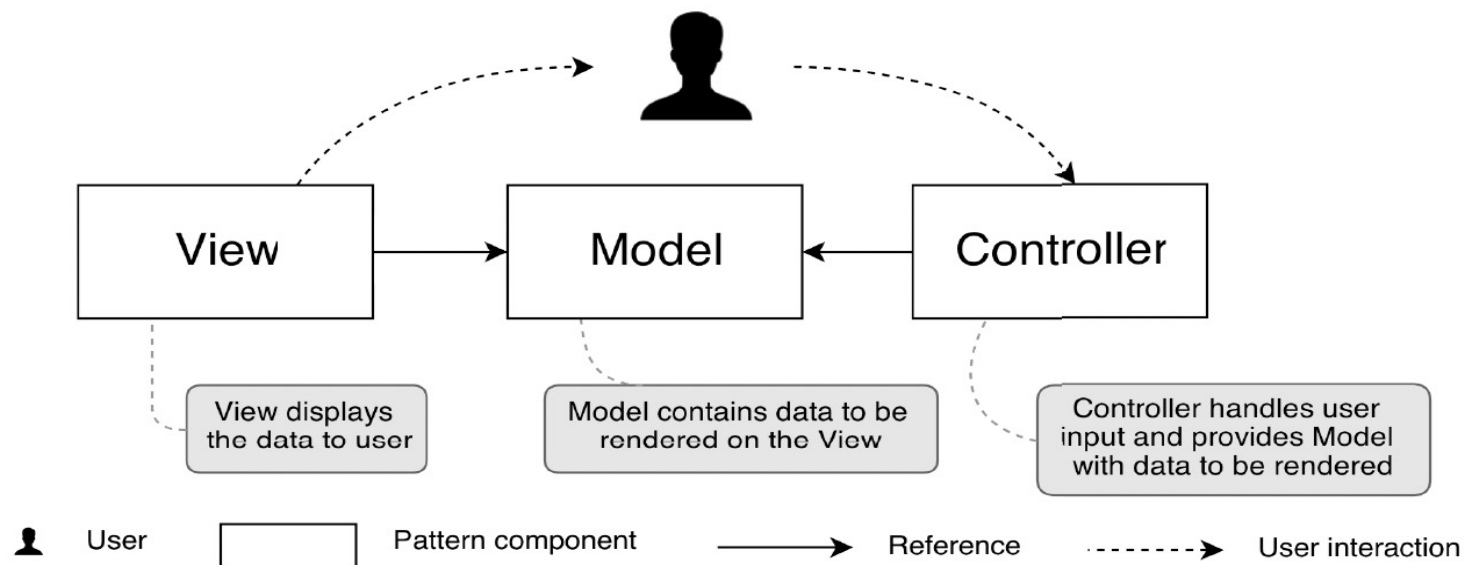
- ✧ Model-View-Controller
- ✧ MVC separates the representation of the application domain (Model) from the display of the application's state (View) & user interaction control (Controller)
- ✧ Smalltalk'80 MVC pattern
 - Smalltalk is a purely OO programming language that was originally created in the 1970s for educational use
 - **Model** – responsible for the *domain data & logic*. Application logic is independent of the presentation of the domain data.
 - **View** – responsible for *displaying model data*
 - **Controller** - Controller is responsible for *handling user input*
 - View & Controller work as a pair allowing the user to interact with the user interface
 - The Model maintains the domain data. Often, the application has one Model & a set of View-Controller pairs working with it. View & controller are loosely coupled.

Smalltalk'80 MVC Pattern



Web MVC Pattern

- ✧ Controller triggers the application logic, handles user input & creates the Model for further rendering. Controller has access to the application domain logic, it is able to read & modify domain data, & run custom calculations
- ✧ Model only stores data that needs to be displayed on a certain View



Web MVC Pattern

- ✧ Supports clear separation of responsibilities of web application logic, which leads to better organized code with improved understandability & maintainability
- ✧ *View* works on the *client* side & *Controller* works on the *server* side
- ✧ Supporting Frameworks
 - ASP.NET
 - Spring
 - Ruby on Rails
 - Grails
 - Angular JS

Application Model

- ✧ Model contains the domain data
- ✧ View is responsible for displaying data
- ✧ Controller handles user input
- ✧ View & Controller do not directly interact with Model
- ✧ Application Model (i.e., the Presentation Model in this pattern), handles View state & provide the means to process user input before submitting it to the Model
- ✧ Application Model simplifies handling view state & provides extra logic that deals with user input before passing it to Model

... Application Model

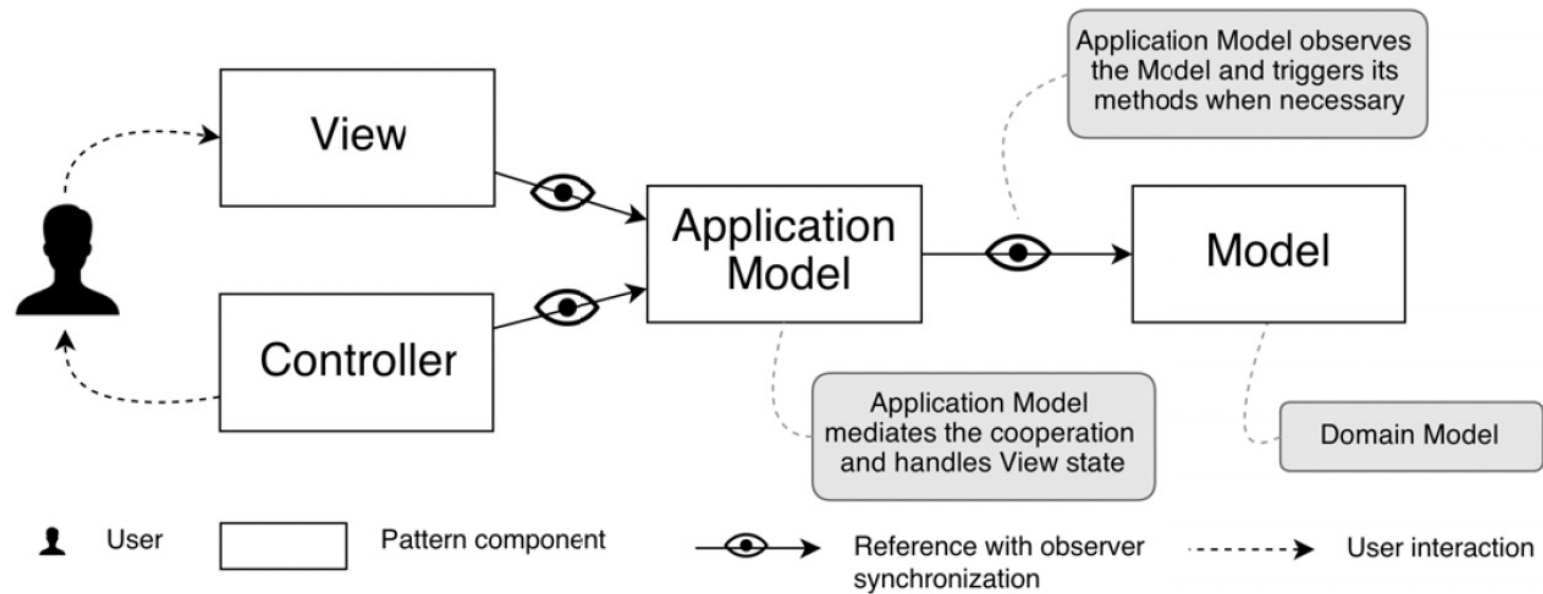


Fig. 4. Application Model pattern

Design Problems

- ✧ To use patterns in your design, you need to recognize that any design problem you are facing may have an *associated pattern* that can be applied
 - Tell several objects that the state of some other object has changed (*Observer pattern*)
 - Tidy up the interfaces to a number of related objects that have often been developed incrementally (*Façade pattern*)
 - Provide a standard way of accessing the elements in a collection, irrespective of how that collection is implemented (*Iterator pattern*)
 - Allow for the possibility of extending the functionality of an existing class at run-time (*Decorator pattern*)

Implementation Issues

- ✧ Focus here is ***not*** on programming, although this is obviously important, but on other implementation issues that are often not covered in programming texts:
 - **Reuse:** Most modern software is constructed by reusing existing components or systems. When you are developing software, you should make as much use as possible of existing code.
 - **Configuration management:** During the development process, you should keep track of the many different versions of each software component in a configuration management system
 - **Host-target development:** Production software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) & execute it on a separate computer (the target system).

Reuse

- ✧ From the 1960s to the 1990s, most new software was developed from scratch, by writing all code in a high-level programming language
 - The only significant reuse of software was the reuse of functions & objects in programming language libraries
- ✧ Costs & schedule pressure mean that this approach became increasingly unviable, especially for commercial & Internet-based systems
- ✧ An approach to development based around the reuse of existing software emerged & is now generally used for business & scientific software

Reuse Levels

✧ Abstraction level

- At this level, you don't reuse software directly but use knowledge of successful abstractions in the design of your software

✧ Object level

- At this level, you directly reuse objects from a library rather than writing the code yourself

✧ Component level

- Components are collections of objects & object classes that you reuse in application systems

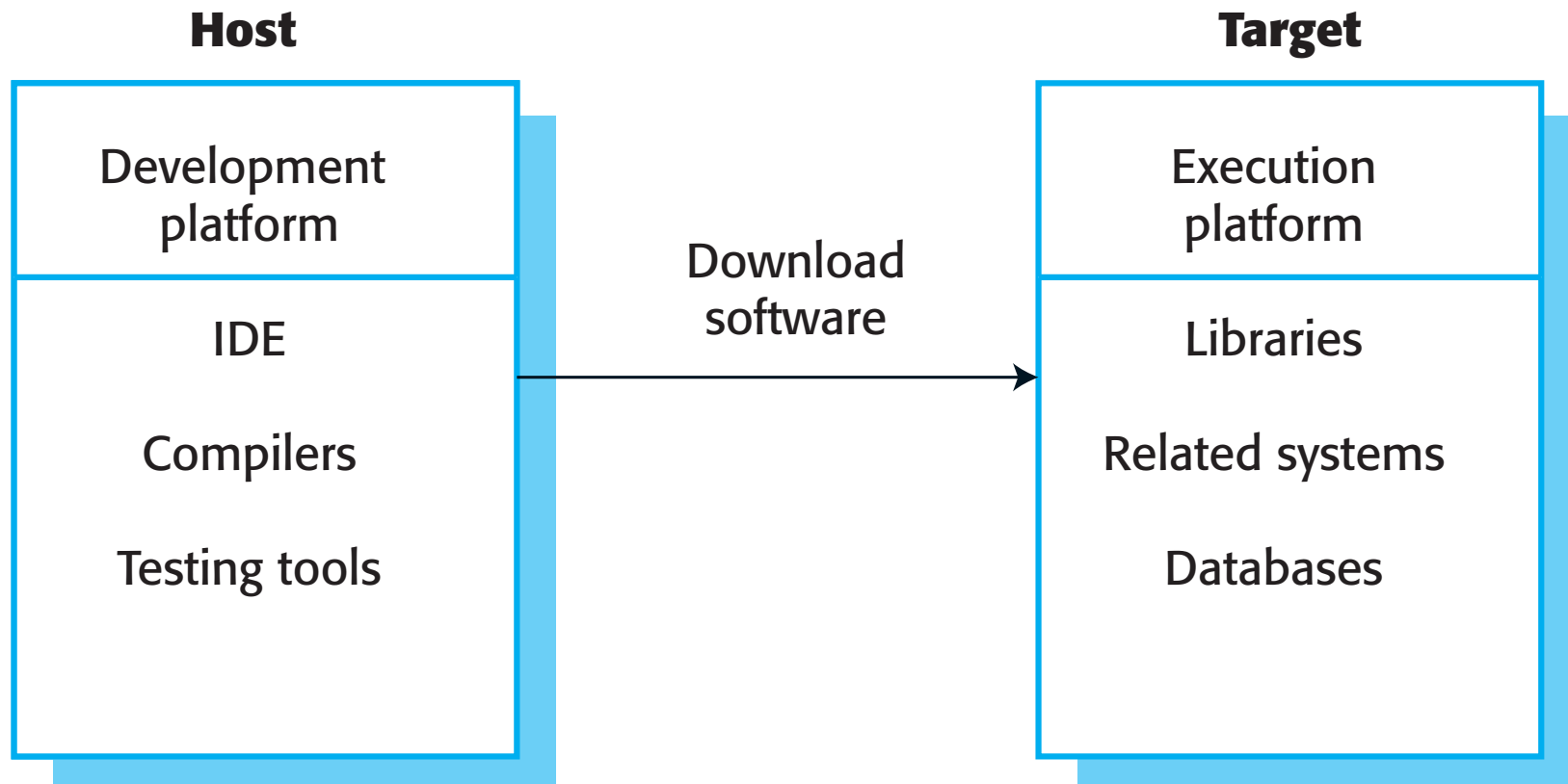
✧ System level

- At this level, you reuse entire application systems

Host-Target Development

- ✧ Most software is developed on one computer (the host), but runs on a separate machine (the target)
- ✧ More generally, we can talk about a development platform & an execution platform
 - A platform is more than just hardware
 - It includes the installed operating system plus other supporting software such as a database management system (DBMS) or, for development platforms, an interactive development environment (IDE)
- ✧ Development platform usually has different installed software than execution platform; these platforms may have different architectures

Host-Target Development



Key Points

- ✧ Software design & implementation are inter-leaved activities. The level of detail in the design depends on the type of system & whether you are using a plan-driven or agile approach.
- ✧ The process of O-O design includes activities to design the system architecture, identify objects in the system, describe the design using different object models & document the component interfaces
- ✧ A range of different models may be produced during an O-O design process. These include *static models* (class models, generalization models, association models) & *dynamic models* (sequence models, state machine models).
- ✧ Component interfaces must be defined precisely so that other objects can use them. A UML interface stereotype may be used to define interfaces.

... Key Points

- ✧ When developing software, you should always consider the possibility of reusing existing software, either as components, services, or complete systems
- ✧ Configuration management is the process of managing changes to an evolving software system. It is essential when a team of people are cooperating to develop software.
- ✧ Most software development is host-target development. You use an IDE on a host machine to develop the software, which is transferred to a target machine for execution.

Reference

- ✧ Syromiatnikov, A. & Weyns, D., 2014, April. A journey through the land of model-view-design patterns. In *2014 IEEE/IFIP Conference on Software Architecture* (pp. 21-30). IEEE.

*** * * End *****